

Chapter 5: Proposed Method

RS-DRL-Based Self-Adaptive System

Thesis Documentation

January 3, 2026

1 Introduction to RS-DRL-Based Self-Adaptive System

Self-adaptive systems operating under uncertainty require not only effective runtime decision making but also clear architectural separation between monitoring, learning, decision, verification, and execution concerns. This section presents the proposed method for integrating Reward Shaping Deep Reinforcement Learning (RS-DRL) with self-adaptive systems, providing a systematic process-oriented refinement of the approach.

Figure 4 illustrates the integration of RS-DRL with a self-adaptive system, highlighting the use of Deep Reinforcement Learning (DRL) to predict suitable adaptation options. The diagram presents a high-level overview of the main components and their interactions. The RS-DRL module is integrated as an upper layer above the MAPE-K loop and is responsible for the continual refinement of the deep reinforcement learning (DRL) model.

To provide a clear and analyzable description of how runtime data is transformed into adaptation decisions, this chapter presents a structured decomposition of the system architecture. Figure 1 introduces a diagram map that organizes the process models presented in this chapter, while Figure 2 provides a high-level integration process overview showing the nine numbered stages of the system operation.

Each surrounding diagram refines a specific concern:

- **D1** details the RS-DRL training and reward shaping process.
- **D2** describes runtime adaptation option prediction.
- **D3** refines the MAPE-K adaptation control logic.
- **D4** presents the adaptation option verification process using runtime models.
- **D5** details monitoring and state construction.

Figure 2 presents the nine numbered stages that characterize the integration process. At the beginning of each episode, the Monitor component collects required runtime data (Stage 1) and forwards them to the RS-DRL module (Stage 2). Based on the received input vectors, the RS-DRL module initiates the training process. Upon completion of training, the updated model is stored in the Knowledge repository (Stage 3). Meanwhile, the Monitor forwards the collected runtime data to the Analyzer for further evaluation (Stage 4). When the Analyzer detects that adaptation goals are violated or are no longer achievable, it notifies the Adaptation Option Predictor, which is embedded in the Planner (Stage 5). The Adaptation Option Predictor retrieves the trained model (Stage 6) and predicts an appropriate adaptation option (Stage 7). The Executor then retrieves the selected option from the adaptation options repository (Stage 8) and applies it to the managed system (Stage 9). This process is repeated for each subsequent adaptation episode.

Diagram Map of RS-DRL Process Models

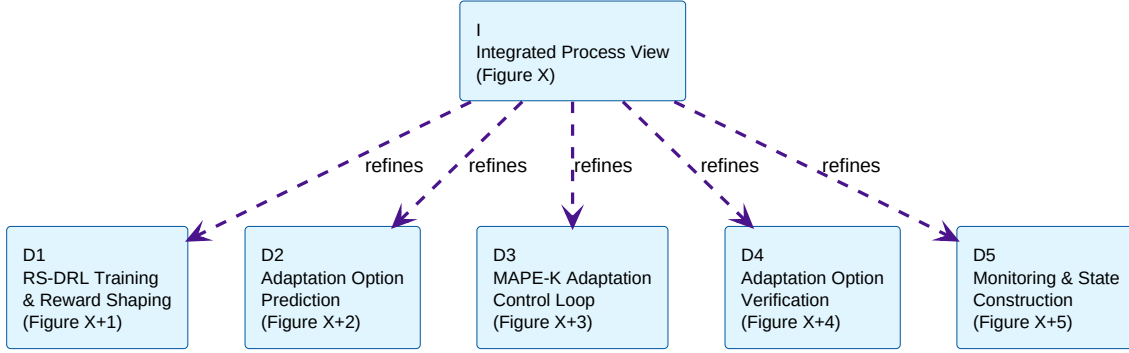


Figure 1: Diagram map of the RS-DRL-based self-adaptive system. The central integration diagram provides a high-level process view, while Diagrams D1–D5 refine the learning, decision-making, verification, control, and monitoring subprocesses presented in this chapter.

2 System Architecture and Integration Overview

2.1 Managed System and Managing System

The proposed architecture consists of two main parts: the **Managed System** and the **Managing System**. The Managed System takes inputs from its environment and produces outputs aligned with user-defined goals. It constantly monitors its state, including system properties, uncertainties, and deviations from desired behavior.

The Managing System oversees the operation of the managed system, employing a MAPE-K loop and an RS-DRL module to facilitate adaptations. The MAPE-K loop provides the control structure for self-adaptation, while the RS-DRL module, integrated as an upper layer above the MAPE-K loop, manages the continual refinement of the deep reinforcement learning model.

2.2 Integration Process Flow

The integration of RS-DRL with the self-adaptive system follows a well-defined process flow that coordinates learning, decision making, verification, and execution. Figure 3 presents the integrated process view that refines the architectural view introduced in the reference model by making control flow, data flow, and feedback loops explicit.

The process begins with the managed system executing its normal behavior and generating runtime data. This data is continuously collected and transformed by the monitoring and state construction process before being forwarded to both the Analyzer and the RS-DRL components. The Analyzer evaluates whether adaptation goals are violated or at risk. When no violation is detected, the system continues normal operation without triggering adaptation.

If adaptation is required, the Planner invokes the adaptation option prediction process, which relies on a previously trained RS-DRL model. Candidate adaptation options are evaluated through runtime model verification, and the resulting quality estimates are used both to justify adaptation decisions and to generate learning feedback. The selected adaptation is then executed on the managed system, closing the control loop.

3 Key Components and Their Responsibilities

This section details the responsibilities of each key component involved in the RS-DRL-based self-adaptive system. The components are presented following the process flow order, from data collection through learning, prediction, control, and verification.

3.1 Monitoring and State Construction

The monitoring process represents the entry point of uncertainty into the system and plays a critical role in enabling effective learning and decision making. The **Sensor Data** component collects raw information from the system and its environment, including sensor readings and system logs.

The **Feature Extractor** processes this raw data to generate a compact state representation that captures essential performance metrics. Raw runtime data is collected from both the managed system and its environment. This data is preprocessed through filtering, normalization, and missing-value handling to reduce noise and stabilize input distributions. Feature extraction then derives goal-relevant attributes while reducing dimensionality.

The resulting features are assembled into a state vector that serves as a contract between monitoring, learning, and planning. The constructed state is forwarded to the Analyzer for goal evaluation, to the RS-DRL module for learning or prediction, and to the Knowledge repository for historical reference.

3.2 RS-DRL Module Components

The RS-DRL module consists of several key components that work together to enable learning and adaptation. The **Adaptation Option Selector** identifies candidate adaptation actions using an epsilon-greedy policy, balancing exploration and exploitation. To predict the most suitable option, it employs a deep Q-network (DQN), which estimates Q-values for each action given the current system state.

The **Adaptation Option Verifier** evaluates the impact of each candidate option by simulating system behavior using runtime models and statistical model checking (Uppaal-SMC), estimating quality attributes such as energy consumption, packet loss, and latency. The **Reward Calculator** assigns rewards based on the verifier’s estimates, thereby guiding the learning process of the RS-DRL agent.

The **Replay Memory** incorporates a novel reward-shaping mechanism. Unlike conventional experience replay, RS-DRL selectively reshapes certain failed experiences and treats them as successful, mimicking aspects of human learning. This mechanism (1) promotes generalization across heterogeneous environments and (2) accelerates learning by transforming past mistakes into informative training samples.

During **Model Training**, batches of experiences are sampled from the replay memory, and the DQN parameters are updated via backpropagation to minimize the loss function, progressively improving Q-value estimation. Finally, the **Trained Models Repository** stores the most recent RS-DRL model, which is utilized in subsequent adaptation episodes.

3.3 Adaptation Option Prediction

The adaptation option prediction process operates at runtime and is strictly non-learning. Unlike the training process, this path is invoked only when adaptation is required and uses a previously trained model without further learning.

Upon notification from the Analyzer, the Adaptation Option Predictor (embedded in the Planner) retrieves the trained RS-DRL model from the Knowledge repository and evaluates the current state against all available adaptation options. The option with the highest expected value is selected, typically using a purely exploitative strategy ($\epsilon \approx 0$) or minimal exploration. The predicted adaptation option is then forwarded to the Executor for enactment. This separation ensures that computationally expensive learning activities do not interfere with runtime responsiveness.

3.4 MAPE-K Adaptation Control Loop

The MAPE-K adaptation control loop provides the orchestration logic for the self-adaptive system. The loop consists of four main components: Monitor, Analyzer, Planner, and Executor, all sharing a common Knowledge repository.

The **Monitor** continuously observes the managed system and its environment, collecting relevant data. The **Analyzer** processes the collected data to identify potential issues or deviations from adaptation goals. When the Analyzer detects that adaptation goals are violated or are no longer achievable, it notifies the Adaptation Option Predictor. The **Planner** activates the Adaptation Option Predictor when issues are detected, utilizing a trained DRL model to predict suitable adaptation options based on the system’s current state. The **Executor** retrieves detailed information from the Knowledge repository and implements the chosen adaptation option, modifying the managed system’s behavior accordingly.

The adaptation process is event-driven rather than continuous. When no violation is detected, the system continues normal operation without triggering adaptation. This design preserves a clear separation between control logic and learned intelligence.

3.5 Adaptation Option Verification

The adaptation option verification process provides semantic grounding and trustworthiness for learning-based decisions. Candidate adaptation options are evaluated using runtime models parameterized with the current system state.

Statistical model checking, implemented via UPPAAL-SMC, is used to estimate the probability of satisfying adaptation goals under uncertainty. Extracted quality metrics are aggregated into either a scalar reward (during training) or a quality estimate (during planning). By mediating between architectural models and learning, verification ensures that rewards and decisions remain aligned with system-level objectives.

4 RS-DRL Training Algorithm

This section provides a detailed technical description of the RS-DRL training algorithm, including the episodic training process, experience replay mechanism, and Q-network updates. The training process is detailed in Figure 5, which shows the complete flow from episode initialization through model storage.

4.1 Training Process Overview

RS-DRL is implemented using an enhanced Deep Q-Network (DQN) algorithm. The training process proceeds in episodes, where each episode represents a complete interaction sequence with the environment. Algorithm 1 presents the RS-DRL algorithm, which aims to train an action-value function $Q(s, a)$ representing the expected cumulative reward. The algorithm integrates with the reward shaping mechanism described in Section 5.

At each step, the current state is used to select an action according to an ϵ -greedy policy. The action is executed in the environment, producing a reward and a subsequent state. Experiences are stored in a replay buffer, and learning is triggered only when sufficient experience has been accumulated.

Key hyperparameters include:

- **Learning rate (η):** Controls how much newly acquired knowledge overrides old information.
- **Discount factor (γ):** Dictates the relevance of future rewards.

Algorithm 1 RS-DRL

Require: Replay buffer D of capacity N , learning rate η , reward shaping factor ρ , discount factor γ , exploration rate ε

Ensure: Trained action-value function Q

```
1: Initialize action-value function  $Q$  with random weights  $\theta$ 
2: Initialize target action-value function  $\hat{Q}$  with weights  $\hat{\theta} \leftarrow \theta$ 
3: for each episode do
4:   Initialize environment and get initial state  $s_0$ 
5:   for each step in the episode do
6:     Select action  $a_t$  using  $\varepsilon$ -greedy policy based on  $Q(s_t, a_t)$ 
7:     Execute action  $a_t$  and observe reward  $r_t$  and next state  $s_{t+1}$ 
8:     Store experience  $(s_t, a_t, r_t, s_{t+1})$  in replay buffer  $D$ 
9:     Sample random minibatch  $B$  of experiences  $(s_i, a_i, r_i, s_{i+1})$  from  $D$ 
10:    Reward Shaping
11:    RewardShaping( $B, \rho$ )
12:    Compute target  $y_i$  for each minibatch transition using reshaped rewards from Equation (2)
13:    Perform a gradient descent step on  $(y_i - Q(s_i, a_i; \theta))^2$  with respect to  $\theta$ , using learning rate  $\eta$ 
14:    Update target network parameters  $\hat{\theta} \leftarrow \theta$  periodically
15:    Decrease exploration rate  $\varepsilon$  according to decay strategy
16:  end for
17: end for
```

- **Exploration rate (ε):** Determines the balance between exploration (trying new actions) and exploitation (choosing best-known actions).

4.2 Q-Network Learning

During training, batches of experiences are sampled from the replay memory. The DQN parameters are updated via backpropagation to minimize the loss function, progressively improving Q-value estimation. Target values are computed using the target network $\hat{Q}$ to stabilize learning, and the target network parameters are periodically synchronized with the primary Q-network.

The exploration rate ε is gradually decreased according to a decay strategy, shifting the agent from exploration to exploitation as training progresses. This allows the agent to initially explore the action space broadly and then converge to exploiting the learned policy.

5 Novel Reward Shaping Mechanism

This section details the core contribution of the proposed approach: the novel reward shaping mechanism that helps the model avoid local optima and find suitable adaptation options efficiently. The mechanism is applied during the training process described in Section 4, specifically during minibatch sampling as shown in Algorithm 1.

5.1 Motivation and Intuition

The reward shaping mechanism is designed to modify the rewards of a subset of experiences within each minibatch to guide the agent’s learning in a way that promotes exploration and accelerates convergence toward better policies. Unlike conventional reward shaping techniques that globally modify the reward function or alter stored experiences, the proposed approach operates exclusively at the minibatch level during learning.

Algorithm 2 Reward Shaping

Require: Minibatch of experiences $B = \{(s_i, a_i, r_i, s_{i+1})\}$, reshaping factor ρ

Ensure: Updated minibatch with reshaped rewards

```
1:  $N \leftarrow |B|$  {Number of experiences in the minibatch}
2:  $K \leftarrow \lceil \rho \cdot N \rceil$  {Number of experiences to reshape based on  $\rho$ }
3: Identify all failed experiences in  $B$  where  $r_i = 0$  {Assume binary rewards}
4: if number of failed experiences  $\geq K$  then
5:   Randomly select  $K$  failed experiences from  $B$ 
6:   for each selected experience  $(s_i, a_i, 0, s_{i+1})$  do
7:     Set  $r_i \leftarrow 1$  {Reshape failure (0) to success (1)}
8:   end for
9: else
10:  Reshape all failed experiences in  $B$  to 1 {If fewer than  $K$  failures exist}
11: end if
12: return updated  $B$ 
```

The mechanism is loosely inspired by ideas from Hindsight Experience Replay (HER) and Soft HER, where failed trajectories are relabeled with alternate goals to simulate successful episodes. However, unlike HER-based approaches, our method does not assume access to a goal-conditioned environment or rely on relabeling mechanisms tied to domain-specific goal definitions.

The core intuition is that failed experiences may still contain valuable exploratory behavior that should not be discarded. By probabilistically reinterpreting certain failures as optimistic successes, the mechanism (1) promotes generalization across heterogeneous environments and (2) accelerates learning by transforming past mistakes into informative training samples.

5.2 Reward Shaping Function

Algorithm 2 presents the reward shaping function that implements the novel mechanism. The function takes a minibatch of experiences and a reshaping factor ρ as input, and modifies the rewards of a subset of failed experiences within the minibatch. This function is called during the training process (line 11 of Algorithm 1).

The reshaping factor $\rho \in [0, 1]$ determines the proportion of minibatch elements to modify. A random index set $K \subseteq \{1, \dots, N\}$ is formed, with its size $|K|$ equal to $\lceil \rho N \rceil$. In practice, the selection of experiences for reshaping is restricted to those that correspond to failures, where failures are defined as transitions where the reward $r_i = 0$ under a binary reward assumption.

Figure 9 isolates the proposed reward shaping mechanism applied during RS-DRL training. After sampling a minibatch from the replay buffer, each experience is evaluated to determine whether it represents a failure transition. Failed experiences are then probabilistically reinterpreted as optimistic successes according to a reshaping probability p . Importantly, reward reshaping is applied only to the sampled minibatch and does not alter the contents of the replay buffer.

5.3 Design Rationale

The design choice of using a constant optimistic reward of 1 is deliberate and domain-agnostic. This approach, instead of relying on handcrafted shaping heuristics or scaled rewards, uses a uniform optimistic signal to bias the value function. This bias encourages the consideration of alternative action paths that might otherwise be ignored due to consistently negative or zero rewards.

The simple yet effective strategy promotes generalization and exploration, particularly in environments like self-adaptive systems where goals are implicit, emergent, or dynamically shifting. The constant value of 1 for the optimistic reward aligns with the characteristics of self-adaptive systems (like DeltaIoT), where adaptation goals are often framed as binary outcomes (1 for satisfying QoS constraints, 0 otherwise).

This design preserves the diversity and integrity of accumulated experiences while mitigating the dominance of failure transitions during early learning stages, thereby accelerating convergence without biasing long-term policy optimization. The reshaped experiences are used to update the Q-network via gradient descent, after which the trained model is stored in the Knowledge repository for later use by the Adaptation Option Predictor.

6 Summary and Traceability

This chapter presented a structured process model of the RS-DRL-based self-adaptive system. Starting from a high-level integration view showing the nine numbered stages (Figure 2), the system was progressively decomposed into monitoring (Section 3.1), learning (Section 3.2), prediction (Section 3.3), verification (Section 3.5), and control (Section 3.4) subprocesses.

Each diagram refines a specific concern while maintaining traceability to the reference architecture (Figure 4). The process models provide a clear foundation for implementation, analysis, and evaluation of the proposed approach under uncertainty. The numbered stages (1–9) are consistently traceable across all diagrams, ensuring that the high-level integration overview (Figure 2) maps correctly to the detailed component diagrams (Figures 4–8).

Together, these figures offer a complete, traceable, and modular representation of the proposed approach, enabling both theoretical analysis and practical implementation of RS-DRL-based self-adaptive systems.

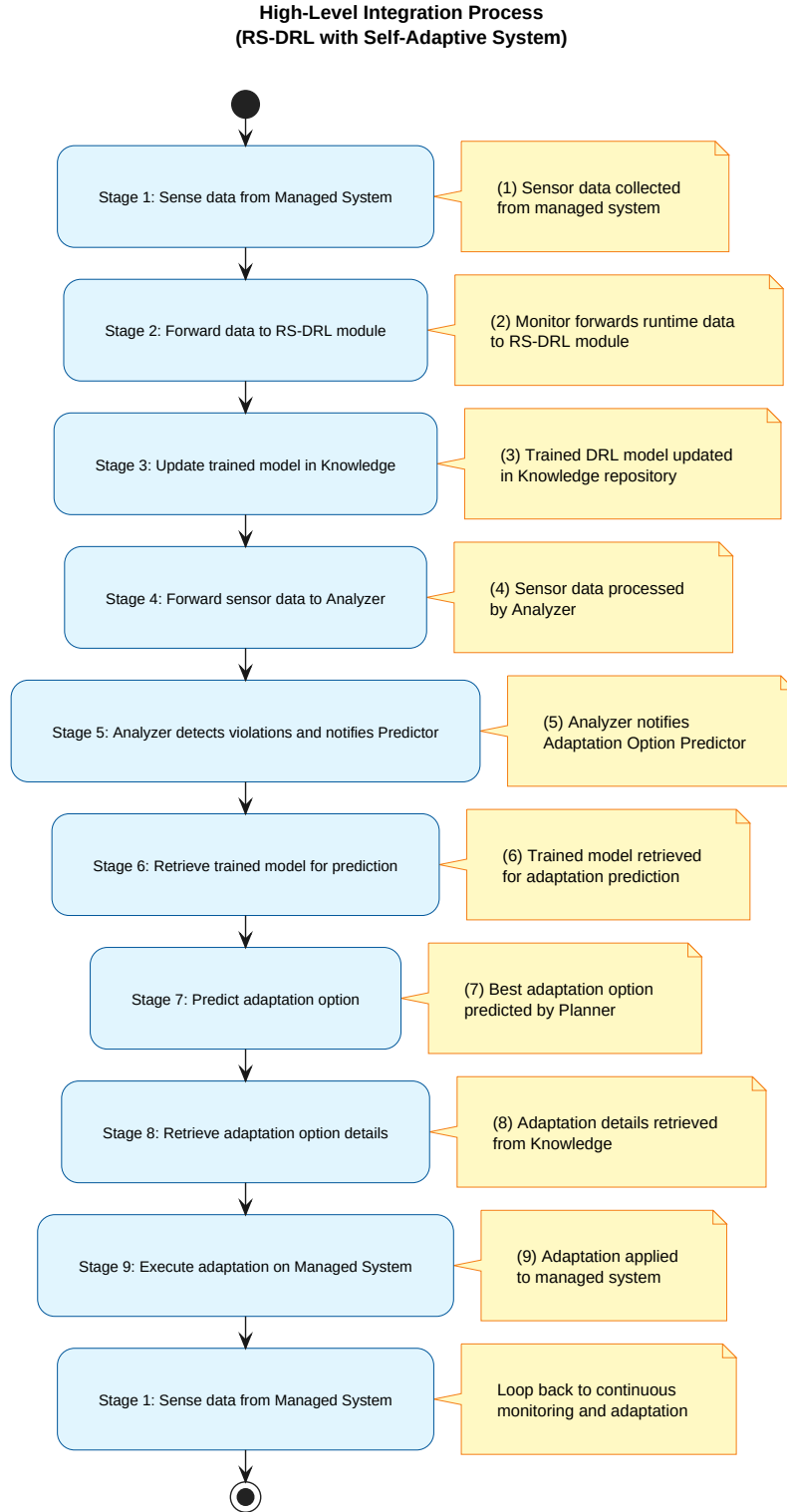


Figure 2: High-level integration process overview showing the nine numbered stages of the RS-DRL-based self-adaptive system. The sequential flow illustrates how data sensing, model updates, analysis, planning, and execution are integrated in a continuous adaptation loop.

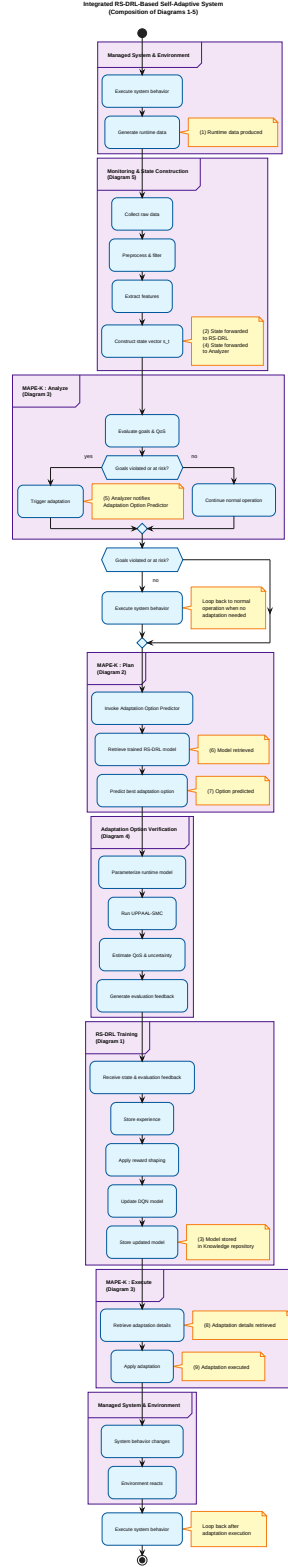


Figure 3: Integrated process view of the RS-DRL-based self-adaptive system. The figure illustrates the closed-loop interaction between monitoring, learning, verification, planning, and execution, and refines the original architectural view presented in Figure 4 of the reference model.

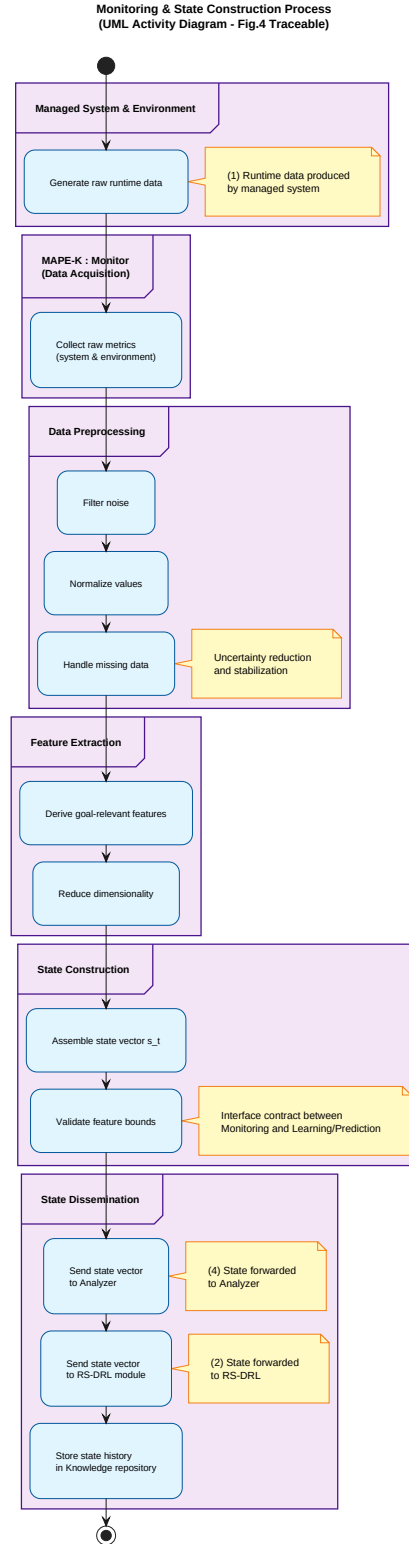


Figure 4: Monitoring and state construction process. Raw runtime data from the managed system and environment is filtered, abstracted, and transformed into a state vector suitable for both RS-DRL learning and runtime adaptation analysis.

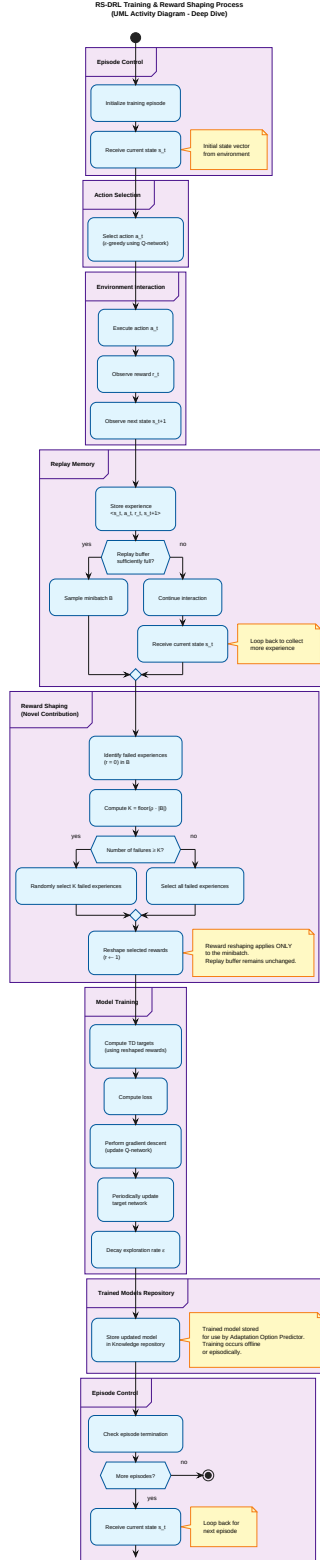


Figure 5: RS-DRL training and reward shaping process. The figure details the episodic interaction with the environment, experience replay, and the novel reward reshaping mechanism applied during minibatch learning.

**Adaptation Option Prediction Process
(Runtime Decision Path - Fig.4 Traceable)**

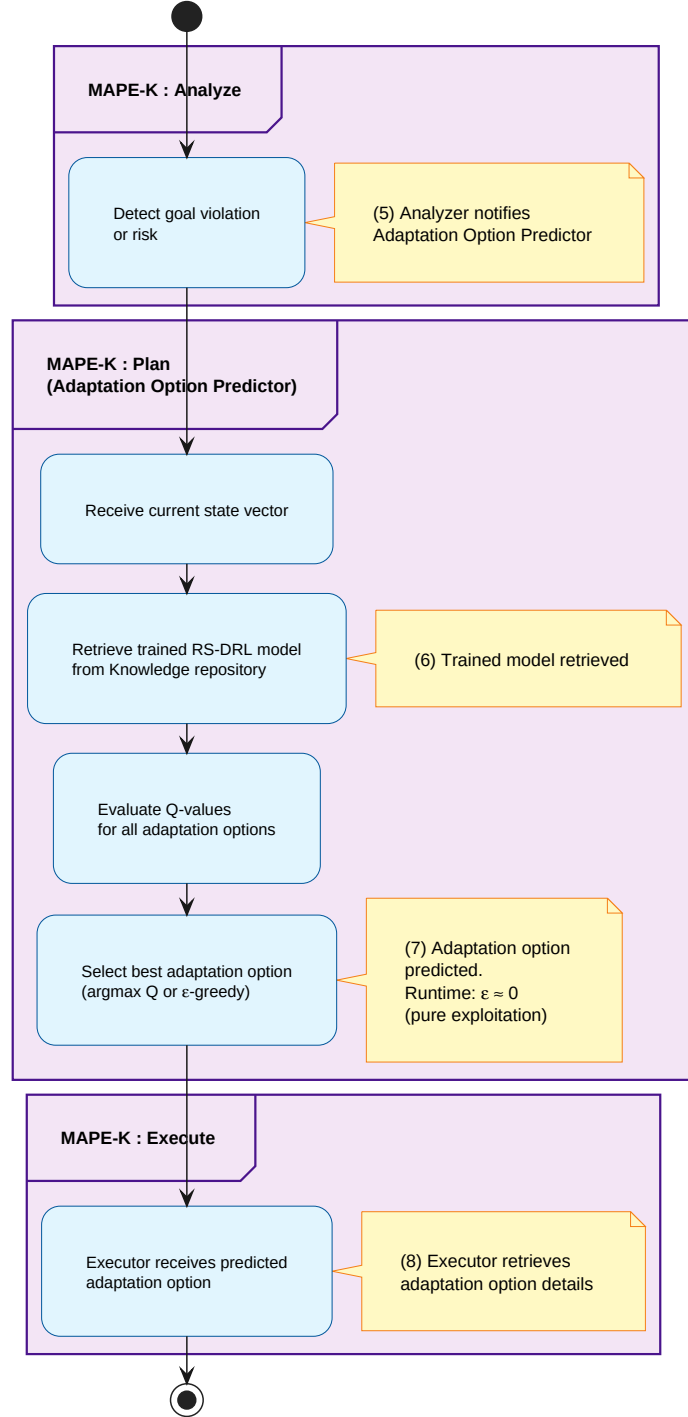


Figure 6: Adaptation option prediction process at runtime. Upon detection of goal violations, the trained RS-DRL model is used to predict the most suitable adaptation option without further learning.

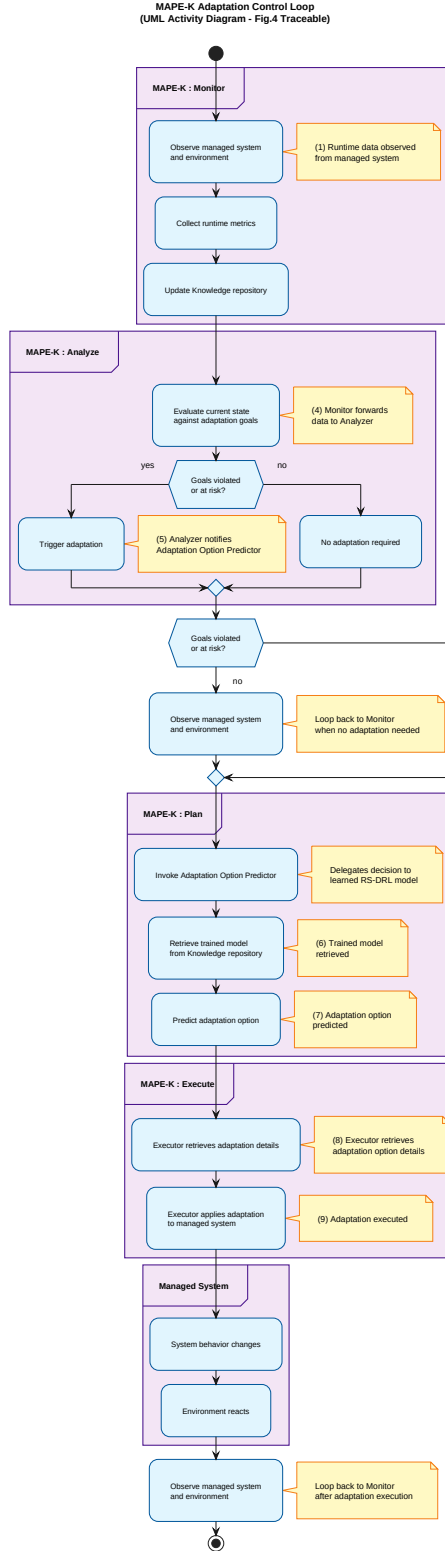


Figure 7: MAPE-K adaptation control loop. The figure illustrates how monitoring and analysis trigger adaptation, how planning delegates decision making to the RS-DRL predictor, and how execution enacts the selected adaptation.

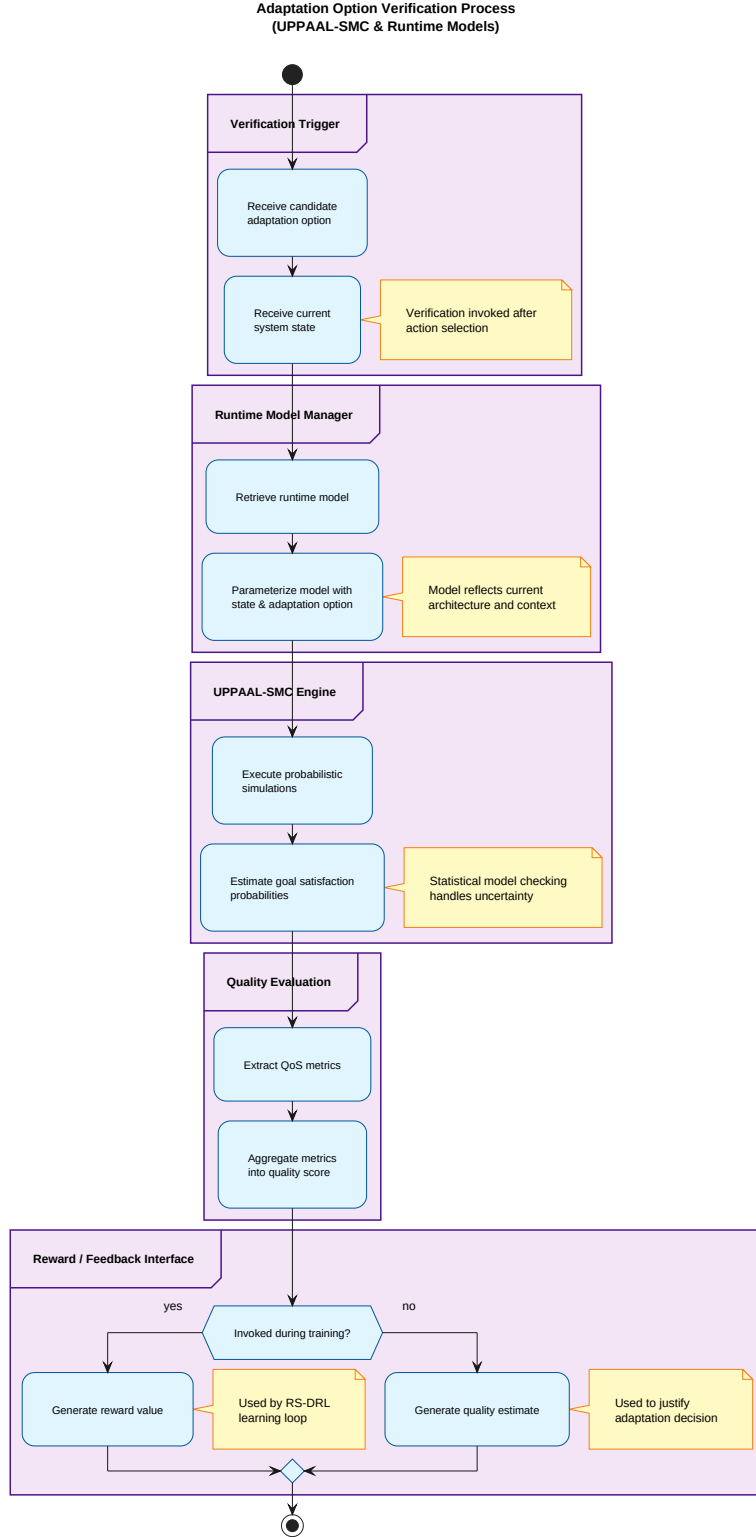


Figure 8: Adaptation option verification using runtime models and statistical model checking. The verification process evaluates the expected quality of adaptation options under uncertainty and provides feedback for learning and decision justification.

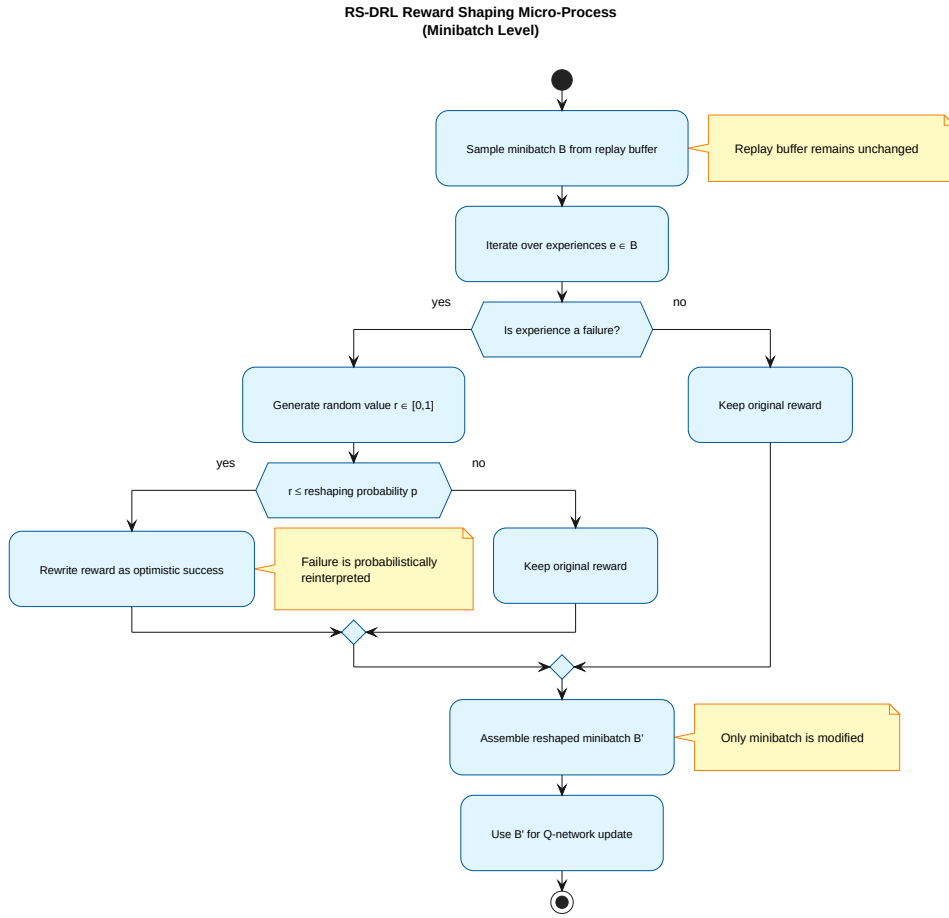


Figure 9: Reward shaping micro-process applied during RS-DRL training. Failed experiences sampled in a minibatch are probabilistically reinterpreted as optimistic successes according to a reshaping probability p . Reward reshaping is applied exclusively to the sampled minibatch and does not modify the replay buffer, preserving experience diversity while accelerating learning convergence.